

TruckLab: Pure Pursuit Path Following Algorithm

Development, Simulation and Validation of the Pure Pursuit Algorithm

Authors: Jakub Kasprzyk, Koen Scholten: k.j.scholten@student.tue.nl

Introduction

In this assignment, you will develop, implement, and test a Pure Pursuit path-following controller for a tractor-trailer vehicle model. The task is designed to combine geometric path tracking concepts with kinematic vehicle modeling, culminating in a working MATLAB simulation.

Exercises 1–9 are preparation exercises and must be completed at home before attending the TruckLab session. Exercise 10 is carried out in the TruckLab, where the controller developed and tested in simulation during Exercises 1–9 will be deployed and evaluated on the actual vehicle hardware. Therefore, make sure that Exercises 1–9 are fully completed and your simulation is working correctly before coming to the lab.

By completing all exercises in this assignment, you will gain hands-on experience in the following areas:

- Understanding the geometric principle behind Pure Pursuit path tracking, including the concept of the lookahead distance and chord-arc relationship.
- Converting target point geometry into a steering command using the single-track vehicle model.
- Implementing discrete-time simulation of vehicle motion using Euler integration.
- Debugging and improving controller behavior through systematic testing and incremental modifications (software-in-the-loop).
- Identifying and correcting angle definition issues (bearing angle calculation) in MATLAB, including quadrant handling and angle wrapping.
- Designing and applying stopping criteria based on proximity to the goal point.
- Structuring simulation scripts and helper functions for modularity and clarity (clean coding).
- Evaluating controller performance through qualitative observation and, later, quantitative metrics such as trajectory and orientation error.
- Hands-on experience in deploying the developed software onto the hardware in the TruckLab.

Upon completing this assignment, you should be able to implement a functional Pure Pursuit controller for a given reference path, validate its performance in simulation, apply systematic debugging to address undesired behaviors, and then deploy it on actual hardware.

Submission: The deliverable for this assignment is a *single PDF report per group* covering Exercises 10.1, 10.4, and 10.5. Exercises 1 until 9 are preparatory and are not handed in, though their implementation is assumed throughout.

- **Scope.** Address exercises 10.1, 10.4, and 10.5, in order, under a clearly labelled heading.

- **Conciseness.** Keep answers compact and to the point: state only what is needed to interpret the result, without padding or repeating information.
- **Figures.** Export all plots in a *vector* format (PDF or EPS), so they stay sharp at any zoom level.
- **Interpretation.** Present no result without explanation. For each figure, table, and fitted number, state what it shows, whether it matches what the kinematic model predicts, where it does not, and which physical effect accounts for the difference.
- **Numerical values.** Report fitted parameters (e.g., a , b , L_{eq}) with units and a sensible number of significant digits, and state the filter settings (`w_pos_vel`, `w_yaw_rate`, `t_start`, `t_end`) used to obtain them.

Pure Pursuit: General Overview

Pure Pursuit is a geometric path tracking algorithm used in mobile robotics to follow a predefined reference trajectory. Rather than directly tracking the entire path, the robot "chases" a point, on the path, located at a fixed distance — the **lookahead distance** (L_d) — ahead of its current position. The algorithm outputs a "steering command" that allows the robot to follow a circular arc towards the selected target point.

In this assignment, you will implement a Pure Pursuit controller for a single-track kinematic model of the Tractor-Trailer system from Assignment 1. You will first simulate the closed-loop behavior of the system using the **Euler integration** method, then deploy your algorithm on scaled RC truck models in the TruckLab setup for experimental validation.

Below is a more implementation-driven overview of the algorithm:

1. **Target Point Selection** Identify the intersection of the lookahead circle centered at the current vehicle position with the reference path.
2. **Target Direction** α Compute the heading difference between the current vehicle orientation and the direction to the target point.
3. **Steering Angle** δ Calculate the required steering angle δ to follow the arc leading to the target point.
4. **Stopping Criterion** If the vehicle is within a threshold distance to the final waypoint, stop it. Otherwise, continue driving with constant forward velocity.
5. Iterative process: Repeat the above steps at each time instance.

Each major section of this assignment corresponds to the steps above and is labeled accordingly.

Exercise 1: Circle-Line Intersection

Finding the **desired** intersection between the search radius and the reference path is the most implementation-heavy part of this assignment. Luckily, it does not take much apart from a bit of geometric formulas (that we will take for granted), a `for` loop, and a couple of nested `if` statements.

This exercise will be done in an auxiliary MATLAB script, called `'circle_intersection_dev.m'` that will allow you to build the intuition and test the logic.

Intermezzo 1: Circle-Line Intersection

An (infinite) line determined by two points (x_1, y_1) and (x_2, y_2) may intersect a circle of radius r and center $(0, 0)$ in:

- two imaginary points (no intersection),
- a degenerate single point (the line is tangent to the circle),
- or two real points (the line intersects the circle at two points).

Those scenarios have been portrayed in Figure 1.

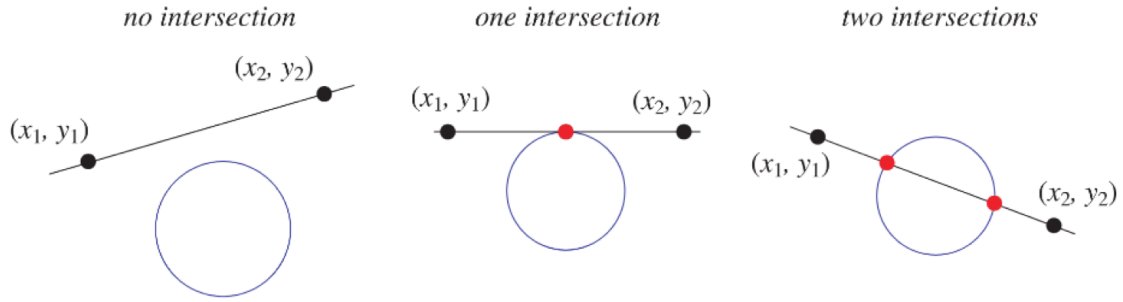


Figure 1: Line-Circle Intersection visualization. [Weisstein \(2025\)](#)

In geometry, a line meeting a circle in exactly one point is known as a **tangent line**, while a line meeting a circle in exactly two points is known as a **secant line** (Rhoad et al., 1984, p. 429).

Define the following:

$$d_x = x_2 - x_1 \quad (1)$$

$$d_y = y_2 - y_1 \quad (2)$$

$$d_r = \sqrt{d_x^2 + d_y^2} \quad (3)$$

$$D = \begin{vmatrix} x_1 & x_2 \\ y_1 & y_2 \end{vmatrix} = x_1 y_2 - x_2 y_1 \quad (4)$$

Let the discriminant be:

$$\Delta = r^2 d_r^2 - D^2 \quad (5)$$

The value of Δ determines the type of intersection between the line and the circle, as summarized below:

$\Delta < 0$	No intersection (imaginary points)
$\Delta = 0$	Tangent (one point of contact)
$\Delta > 0$	Two real intersection points (secant)

Then, the points of intersection between the line and the circle are given by:

$$x = \frac{D d_y \pm \text{sgn}^*(d_y) d_x \sqrt{\Delta}}{d_r^2} \quad (6)$$

$$y = \frac{-D d_x \pm |d_y| \sqrt{\Delta}}{d_r^2} \quad (7)$$

The function $\text{sgn}^*(x)$ is defined as:

$$\text{sgn}^*(x) = \begin{cases} -1 & \text{for } x < 0 \\ 1 & \text{otherwise} \end{cases}$$

Exercise 1.1

[circle_intersection_dev.m]

Implement the $\text{sgn}^*(x)$ function definition in `sgn_star(x)`.

Exercise 1.2

[circle_intersection_dev.m]

Implement equations from Equation 1 to Equation 5.

Exercise 1.3

[circle_intersection_dev.m]

The intersections occur only if the discriminant is greater than or equal to zero ($\Delta \geq 0$). Implement Equation 6 and Equation 7 inside the `if delta >= 0` statement in the script. Utilize the $\text{sgn}^*(x)$ function defined in Exercise 1.1.

When solving for the intersection between a circle and a line segment, the solution may yield one or two points. However, these intersection points are computed assuming the line is infinite.

To determine whether these points lie on the finite line segment connecting (x_1, y_1) and (x_2, y_2) , we must check whether they fall within the bounding box defined by the segment's endpoints.

In our logic for choosing valid intersections, we need to encompass four edge cases, portrayed in Figure 2.

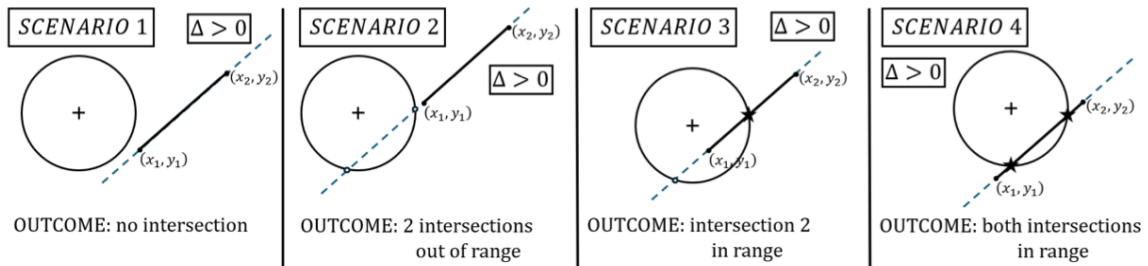


Figure 2: Possible scenarios of line-circle intersection.

Exercise 1.4

[circle_intersection_dev.m]

Fill out the `'intersec1_in_range'` and `'intersec2_in_range'` conditions. Analyze the `'if'` statements found below exercise 1.4 in `circle_intersection_dev.m`, and use the extrema: `minX`, `maxX`, `minY`, and `maxY`, declared above.

Exercise 1.5

Verify if your code is working as intended. Try all different edge cases, including vertical and horizontal lines (Hint: those will not work as intended if you used a MATLAB 'sign' function).

Exercise 2: Target Point Selection

In this exercise, you will develop a function called 'target_search'. Our target point is nothing more than an intersection of the search radius and the path, if we view the two consecutive waypoints as points defining a line. The task of finding the target point becomes the problem from **Exercise 1**, solved continuously as the vehicle is moving.

Here is the pseudocode of the 'target_search' function to allow you to better understand what we will set out to do in Exercise 2.

Algorithm 1 Target Point Search Logic [simplified pseudocode]

```

1: for  $i = \text{lastFoundIndex}$  to  $\text{length}(\text{path}) - 1$  do
2:    $\triangleright$  Compute intersections for segment  $i \rightarrow i + 1$  using Exercise 1
3:    $\triangleright$  Results:  $\text{intersection\_1}$ ,  $\text{intersection\_2}$ ,  $\text{intersec1\_in\_range}$ ,  $\text{intersec2\_in\_}$ 
       $\text{range}$ ,  $\Delta$ 
4:   if  $\Delta \geq 0$  then
5:     if  $\text{intersec1\_in\_range}$  OR  $\text{intersec2\_in\_range}$  then
6:       if  $\text{intersec1\_in\_range}$  AND  $\text{intersec2\_in\_range}$  then
7:         if  $\|\text{intersection\_1} - p_{i+1}\|_2 < \|\text{intersection\_2} - p_{i+1}\|_2$  then
8:            $\text{target\_point} \leftarrow \text{intersection\_1}$ 
9:         else
10:           $\text{target\_point} \leftarrow \text{intersection\_2}$ 
11:        end if
12:      else
13:        if  $\text{intersec1\_in\_range}$  then
14:           $\text{target\_point} \leftarrow \text{intersection\_1}$ 
15:        else
16:           $\text{target\_point} \leftarrow \text{intersection\_2}$ 
17:        end if
18:      end if
19:      if  $\|\text{target\_point} - p_{i+1}\|_2 < \|\text{currentPose} - p_{i+1}\|_2$  then
20:         $\text{lastFoundIndex} \leftarrow i$ 
21:        break
22:      else
23:         $\text{lastFoundIndex} \leftarrow i + 1$     (continue)
24:      end if
25:    else
26:       $\text{target\_point} \leftarrow p_{i+1}$     (fallback on this segment)
27:    end if
28:  end if
29: end for

```

Within the functionality of 'target_search' we aim to capture three main objectives:

1. Selection of the preferred intersection out of valid candidates. [Algorithm 1: line 6-18] → Exercise 2.3 & 2.4
2. Limiting the search to non-traversed segments of the reference path only. If the vehicle is closer to the end of the segment than the found target point, limit the search to the next segment onwards. [Algorithm 1: line 19-24] → Exercise 2.5
3. Falling back to the end of the current segment if no intersection is found (a design choice). [Algorithm 1: line 26] → Exercise 2.6

In the following exercises you will implement all of those functionalities.

Exercise 2.1: Euclidean distance [`\+helpers\euclidean_distance.m`]

Implement the L2 norm in the 'euclidean_distance.m' function, found in the `helpers` folder.

Exercise 2.2: In-range check (reuse Ex. 1.4) [`\+helpers\target_search.m`]

In the 'target_search.m' function definition, fill in the 'intersec1_in_range' and 'intersec2_in_range' boolean variables, using the `minX`, `maxX`, `minY` and `maxY` extrema. Notice that the extrema are now defined by the consecutive waypoints picked iteratively from the reference path, as we loop over it during the target point search.

Exercise 2.3: Pick the preferred intersection (when both valid) [`\+helpers\target_search.m`]

Write an 'if' statement that chooses the preferred intersection out of two valid ones, when both lie in the bounding box defined by the two consecutive waypoints.

Exercise 2.4: Pick the desirable intersection (when only one valid) [`\+helpers\target_search.m`]

Write an 'if' statement that chooses a valid intersection, when only one of the intersections is valid (i.e., it is within the range determined by the extrema of the two consecutive waypoints).

Exercise 2.5: Search over non-traversed segments [`\+helpers\target_search.m`]

Limiting the target search to only non-traversed segments of the reference path is crucial for the robustness of the algorithm when dealing with closed-loop trajectories. The `lastFoundIndex` variable marks the current valid segment. If the vehicle's position is closer to the end of this segment than the selected target point, advance the search to the next segment by incrementing `lastFoundIndex`.

Here is the skeleton of the 'if' statement to help you out.

```
1 if helpers.euclidean_distance(target_point, path(i+1,:)) < ...
2     helpers.euclidean_distance([currentX currentY], path(i+1,:))
3     lastFoundIndex = ...; % stay on this segment
4     break % good target point found
5 else
6     lastFoundIndex = ...; % move search forward to the next segment and KEEP
7     LOOPING
8 end
```

Exercise 2.6: Fallback target point [`\+helpers\target_search.m`]

If no valid intersection is found, set the target point to the end of the current segment.

(Hint: `lastFoundIndex` refers to the segment index, while `path` is indexed by waypoints. For example, `lastFoundIndex = 1` would point to the first segment of the reference path, but that segment would be defined by two points: `path(1,:)` and `path(2,:)`)

Exercise 3: Target Direction

At this stage of the assignment, you have successfully dealt with the most implementation-heavy part — determining the target point in all foreseeable scenarios, while ensuring the search is limited to non-traversed segments of the reference path. Well done!

From now on, the remaining steps of the Pure Pursuit algorithm should be less algorithmic and rely on straightforward trigonometry.

The absolute position of the target point cannot act as a direct actuation to our kinematic model. The vehicle is steered via the steering angle δ . In this exercise, you will compute the target direction α , assuming the rear axle of the vehicle will travel along a constant-radius arc from its current position to the target point.

Once we have α , we will use it in Exercise 4 to compute the corresponding steering angle δ .

Intermezzo 2: Target Direction Derivation

The target direction α is defined as the angular difference between the vehicle's current orientation and the bearing from the vehicle's rear axle to the target point.

Let (x_1, y_1) denote the current position of the rear axle and $(x_{\text{target}}, y_{\text{target}})$ the coordinates of the target point. The slope m_{target} of the line connecting these two points is:

$$m_{\text{target}} = \frac{y_{\text{target}} - y_1}{x_{\text{target}} - x_1} \quad (8)$$

From elementary trigonometry, the slope of a line is also related to its orientation θ with respect to the X -axis by:

$$m_{\text{target}} = \tan \theta \quad (9)$$

where θ is measured counterclockwise from the X -axis.

By inverting (9) and substituting (8), the bearing angle θ is:

$$\theta = \arctan \left(\frac{y_{\text{target}} - y_1}{x_{\text{target}} - x_1} \right) \quad (10)$$

The target direction α is then obtained by subtracting the vehicle's current heading ψ_1 from the bearing θ :

$$\alpha = \theta - \psi_1 \quad (11)$$

which, in explicit form, yields:

$$\alpha = \arctan \left(\frac{y_{\text{target}} - y_1}{x_{\text{target}} - x_1} \right) - \psi_1 \quad (12)$$

This α represents the angular deviation that the vehicle must correct via its steering input to follow a constant-radius arc towards the target point.

Practical Note: Why atan2 Matters

Up to now, we derived the target direction α using the slope of the line between the vehicle's rear axle and the target point, followed by applying the standard inverse tangent function $\arctan$. While mathematically correct, this approach introduces a serious issue in implementation:

- The standard $\arctan(y/x)$ only returns angles in the interval $[-\pi/2, \pi/2]$.
- This means that two points with the same slope m but located in different quadrants (e.g., quadrant II vs. quadrant IV) are indistinguishable.
- In practice, this led to the "reluctance to turn left" behavior observed during testing — the vehicle consistently misinterpreted targets lying in quadrants II or III.

To solve this, MATLAB provides the `atan2(Y,X)` function, which computes the four-quadrant inverse tangent. Unlike `atan`, it takes both the numerator and denominator of the slope separately, and returns an angle in the full closed interval $[-\pi, \pi]$. This guarantees that bearings to target points are uniquely identified across all four quadrants.

Normalization of α

Even when using `atan2`, the resulting difference $\alpha = \theta - \psi_1$ must be normalized. This is because:

- Both θ and ψ_1 are wrapped to $[-\pi, \pi]$.
- Their difference can exceed this interval (e.g., subtracting -179° from $+179^\circ$).
- For steering, we always require α in the range $(-\pi, \pi]$, representing the *shortest angular deviation* between current heading and target bearing.

Normalization can be implemented using a modulo operation:

$$\alpha = \text{mod}(\alpha + \pi, 2\pi) - \pi$$

which wraps any value of α back into the desired interval.

Exercise 3.1: Target Direction α implementation with `atan2()` [`\+helpers\target_direction.m`]

In the function `target_direction` found in the `helpers` folder, implement the target direction α calculation discussed in Intermezzo 2 and the Practical Note. **Please use `atan2()` and remember to normalize the obtained α , so that $\alpha \in (-\pi, \pi]$.**

Exercise 4: Steering Angle

After calculating the target direction in Exercise 3, we can proceed to convert it to a steering angle, which will be the output of the developed Pure Pursuit controller.

Intermezzo 3: Target Direction to Steering Angle Conversion

In the Pure Pursuit framework, the vehicle is assumed to follow the *minor circular arc* subtended by the chord connecting the current position of the rear axle to the selected target point. The length of this chord corresponds to the lookahead distance L_d .

The radius R of the circular arc is related to the target direction angle α via the chord length formula:

$$R = \frac{L_d}{2 \sin \alpha} \quad (13)$$

The curvature κ of the path is defined as the reciprocal of the radius:

$$\kappa = \frac{1}{R} = \frac{2 \sin \alpha}{L_d} \quad (14)$$

For a single-track kinematic model, the steering angle δ is related to the curvature by:

$$\delta = \arctan(L_1 \kappa) \quad (15)$$

where L_1 denotes the wheelbase (the distance between the front and rear axles).

Substituting (14) into (15) yields the steering control law:

$$\delta = \arctan\left(\frac{2L_1 \sin \alpha}{L_d}\right) \quad (16)$$

Exercise 4.1: Steering Angle implementation [\+helpers\TargetDir2SteerAng.m]

In the 'TargetDir2SteerAng.m' function, found in the `helpers` folder, implement the steering angle, Equation (16).

Exercise 5: Simulation

Now that we have the core of the Pure Pursuit controller implemented, it is a good time to test the current state of the developed algorithm. For that we will use the 'pure_pursuit_dev_simulation.m' MATLAB script.

Please take a closer look at the 'for' loop found in the 'Simulation Loop' section of the script. This section is the core of the system state simulation. We can see the individual steps of the Pure Pursuit algorithm being iteratively executed in it:

Step 1: Select the target point and ensure forward search by incrementing the `lastFoundIndex` variable if needed.

Step 2: Calculate target direction α .

Step 3: Convert the target direction α into a steering angle input δ_{in} .

Step 4: Feed the steering angle input δ_{in} into the kinematic model to obtain derivatives of vehicle position and orientation.

Step 5: Integrate the obtained derivatives.

Feel free to inspect the `helpers.kinematic_model` function, which is the implementation of the single-track model from Assignment 1, now in MATLAB Function block form.

To simulate the state of the system in a discrete manner, we will use the well-known Euler's method, valued for its simplicity.

Intermezzo 4: Euler's Method for State Propagation

We propagate the vehicle state using the *forward (explicit) Euler* integrator. Let the continuous-time model be

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t)), \quad \mathbf{x}(t) \in \mathbb{R}^4,$$

with the state chosen as

$$\mathbf{x} = \begin{bmatrix} x_1 & y_1 & \psi_1 & \psi_2 \end{bmatrix}^\top, \quad \dot{\mathbf{x}} = \begin{bmatrix} \dot{x}_1 & \dot{y}_1 & \dot{\psi}_1 & \dot{\psi}_2 \end{bmatrix}^\top.$$

Since our simulation is discrete, we reformulate the problem in discrete time. Given a sampling period Δt , the state update from step k to $k + 1$ is

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k) \Delta t,$$

where $\mathbf{f}(\mathbf{x}_k, \mathbf{u}_k)$ denotes the vector field (i.e., the RHS of the ODE, or dynamics function — a.k.a. the state rate/derivative) evaluated at the current state $\mathbf{x}_k$ and input $\mathbf{u}_k$.

Mapping to the script. In the code, the kinematic model returns the derivatives $\dot{x}_1, \dot{y}_1, \dot{\psi}_1, \dot{\psi}_2$ for the current state and input:

```
[x1_dot, y1_dot, psi1_dot, psi2_dot] ← helpers.kinematic_model(  
    V1, delta_in, x_1, y_1, psi_1, psi_2).
```

The Euler step is then implemented as

```
state(k+1) ← state(k) + Δt · [x1_dot y1_dot psi1_dot psi2_dot],
```

Note: Forward Euler is first-order accurate (local truncation error $\mathcal{O}(\Delta t^2)$, global error $\mathcal{O}(\Delta t)$). Choose Δt sufficiently small for stability and accuracy.

Practical Note: Before diving into the implementation of the Euler integration step, a few words about the already-defined state vector — or rather, the state **array**. For memory and computational efficiency, the state has been defined as a matrix `X = zeros(length(t_vec), 4);`. In that form, the columns of the matrix represent the individual states of the system: x_1, y_1, ψ_1 and ψ_2 respectively. The matrix has a number of rows equal to the number of time steps in the simulation. Hence, each row of this matrix contains a full state vector for a given time step.

Exercise 5.1: Euler's integration implementation [pure_pursuit_dev_simulation.m]

In the provided 'pure_pursuit_dev_simulation.m' MATLAB script, inside the `for k = 1:length(t_vec)-1` loop, implement the discrete integration of the derivatives previously obtained from the kinematic model.

Exercise 5.2: Run simulations

Now we can finally test the developed Pure Pursuit algorithm. To do so we will load different reference paths — `path_string = "paths/path_name.mat";` — and alter initial conditions: `p.init.x10, p.init.y10, p.init.psi10` and `p.init.psi20`.

Please simulate the following experiments:

- "paths/dev_right_turn_path.mat"; initial conditions: $(0, 0, \frac{\pi}{2}, \frac{\pi}{2})$
- "paths/dev_right_turn_path.mat"; initial conditions: $(-0.5, 0, \frac{3\pi}{5}, \frac{3\pi}{5})$
- "paths/dev_right_turn_path.mat"; initial conditions: $(0.5, 0, \frac{2\pi}{5}, \frac{2\pi}{5})$
- "paths/dev_left_turn_path.mat"; initial conditions: $(-0.5, 0, \frac{3\pi}{5}, \frac{3\pi}{5})$

At this stage, in all of the above experiments the Pure Pursuit controller should behave in a desirable manner, tracking the reference paths regardless of the given initial conditions.

The only remaining undesired behavior occurs once the reference waypoints are exhausted: the controller continues at constant velocity and departs from the path. A stopping criterion to prevent this behavior is introduced in Exercise 6.

Exercise 6: Stopping criterion

[pure_pursuit_dev_simulation.m]

With the correct behavior of the Pure Pursuit controller now in place, the final step is to add a stopping criterion to the implementation.

Modify the 'pure_pursuit_dev_simulation.m' script to include a simple proximity-based stopping condition. If this distance falls below a chosen threshold (e.g., 0.2 m), set the vehicle velocity to zero and terminate the simulation loop.

This will ensure the simulation ends automatically once the vehicle reaches the goal.

Exercise 7: Final benchmark simulation

Since the developed controller seems to be robust to all the previously simulated scenarios, it is time to test it on a closed-loop route.

Exercise 7.1: Plotting the results

[pure_pursuit_dev_simulation.m]

To make the experiments more informative, plot the target direction α , steering angle δ , and orientation of the tractor ψ_1 over simulation time. Create a 3-by-1 subplot. Test the plotting on the last two experiments from Exercise 5.2. Build up your intuition about the vehicle heading angle and how it evolves over time w.r.t. the reference path.

Exercise 7.2: Figure eight route testing

In the 'pure_pursuit_dev_simulation.m' script, load the "paths/dev_figure_eight_path.mat" path, and run experiments with the following initial conditions:

- $(-8, 3, 0, 0)$
- $(0, 0, \frac{\pi}{4}, \frac{\pi}{4})$
- $(4, -4, \frac{\pi}{2}, \frac{\pi}{2})$
- $(4, -4, \frac{3\pi}{2}, \frac{3\pi}{2})$

In each experiment, analyze the target direction α , steering angle δ , and orientation of the tractor ψ_1 . Did we manage to achieve desirable behavior of the Pure Pursuit controller in all scenarios? If not, which ones were problematic? How would you improve the controller performance?

Exercise 8: Quantitative performance evaluation

As engineers we can rely on qualitative evaluation in the early stages of the development process, as we have done so far. But eventually, we need to present quantitative evaluation using some KPIs (key performance indicators).

Intermezzo 5: Trajectory and Rotation Error Metrics

To evaluate tracking performance quantitatively, we will use two common KPIs:

- **Absolute Trajectory Error (ATE):** Measures the Euclidean distance between the vehicle's actual position and the reference position at each timestep:

$$\text{ATE}(k) = \sqrt{(x_1(k) - x_{\text{ref}}(k))^2 + (y_1(k) - y_{\text{ref}}(k))^2} \quad (17)$$

- **Root square ATE:** To capture the trend of the path-following algorithm, the running RMSE of ATE can be used. Running RMSE is computed per time step, each time evaluated w.r.t. the elapsed time:

$$\text{RMSE_ATE}(i) = \sqrt{\frac{1}{i} \sum_{k=1}^i (\text{ATE}_k)^2} \quad (18)$$

ATE focuses on point-wise positional accuracy, while RMSE-ATE captures the overall trend of path-following performance. The last value of the running RMSE-ATE can act as a single-value evaluator of controller performance — a KPI.

Exercise 8.1: ATE implementation

[+helpers\ATE_error.m]

Inside the 'ATE_error' function definition, implement the Absolute Trajectory Error (ATE), Equation (17).

Exercise 8.2: RMSE-ATE implementation

[+helpers\ATE_error.m]

Inside the 'ATE_error' function definition, implement the running RMSE-ATE, Equation (18).

Exercise 8.3: Lookahead distance influence on tracking performance

Run the `ex8_lookahead_dist_analysis.m` MATLAB script, in which the lookahead distance has been swept over a range of values for the simulation from Exercise 5 using `dev_left_turn_path.mat`. You should see two figures:

- Figure 101 with the reference path and traversed routes resulting from different L_d ,
- Figure 102 with ATEs and their RMSE plotted over time, again for different L_d .

What kind of behavior of the Pure Pursuit algorithm can you observe in Figure 101 w.r.t. the given L_d ? How does this behavior relate to the tracking performance captured in Figure 102?

Exercise 9: Simulink integration

So far, the entire simulation has been carried out in MATLAB for convenience and control. The TruckLab setup, however, requires porting the developed controller into Simulink.

To make this transition as smooth as possible, you are provided with the following:

- Simulink models: `PurePursuit_Tractor_Home.slx` and `PurePursuit_Tractor_Lab.slx`
- MATLAB scripts to run them: `TruckLab_A2_Home.m` and `TruckLab_A2_Lab.m`

The only part of the models that requires implementation is the 'pure_pursuit' function definition, located inside the Pure Pursuit Controller MATLAB Function block.

The `_Home` versions are designed for testing and practice, allowing you to adapt your controller to the Simulink workflow at home. Once everything works, switching to the `PurePursuit_Tractor_Lab.slx` model should be as simple as copying the 'pure_pursuit' function definition from the home example to the Pure Pursuit Controller MATLAB Function block in `PurePursuit_Tractor_Lab.slx`. After that, you will be ready for the hands-on assignment in the Automotive Technology Laboratory.

Exercise 9.1: At Home Simulink integration [`PurePursuit_Tractor_Home.slx`]

Port your developed Pure Pursuit controller to the function definition found in the Pure Pursuit Controller MATLAB Function block of `PurePursuit_Tractor_Home.slx` Simulink model. **Make sure to copy/paste all the helper functions as sub-functions at the end of the script.** This will make potential in-lab debugging easier.

Exercise 9.2: Laboratory preparation [`PurePursuit_Tractor_Lab.slx`]

Copy/paste the working function definition from the Pure Pursuit Controller MATLAB Function block of `PurePursuit_Tractor_Home.slx` Simulink model to the Pure Pursuit Controller MATLAB Function block of `PurePursuit_Tractor_Lab.slx`.

Exercise 10: Hands-on Truck Lab session

This part of the assignment is meant to be completed in-person in the Automotive Technology Laboratory located in Gemini North. Before coming to the hands-on session, students are required to have a working Pure Pursuit algorithm implemented in the provided `PurePursuit_Tractor_Lab.slx` Simulink model. This can be easily verified by students upon completion of Exercise 9.1.

The technician will be present during the hands-on session, but they are only responsible for solving setup-related problems, not answering questions regarding the merit of the assignment; those questions are meant to be asked during the Guided Self-Study sessions.

Exercise 10.1: Experimental preparation [TruckLab_A2_Lab.m + PurePursuit_Tractor_Lab.slx]

The purpose of this exercise is to get prepared for the experiments in Exercise 10.2 and 10.3. Before starting the experiments, verify that:

- the OptiTrack position and orientation can be received on the computer by running `getRigidBodyPose(ID)`, where `ID` is the truck ID of a truck in the TruckLab (`getRigidBodyPose.m` is located in the assignment folder);
- `PurePursuit_Tractor_Lab.slx` is logging and controlling your truck;
- `TruckLab_A2_Lab.m` runs without errors;
- your best lookahead distance from Exercise 8.3 is implemented in `TruckLab_A2_Lab.m`;
- the steering angle offset in `PurePursuit_Tractor_Lab.slx` is corrected (if the truck has one).

In your report:

- state which truck you used;
- explain how you determined the steering angle offset (state clearly whether it was obtained from a previous lab session or measured in this one);
- if your truck has a steering angle offset, explain how you corrected for it; if not, explain how you would correct for it in principle;
- defend your choice for the look-ahead distance.

Important: After running each experiment, save the recorded OptiTrack data (i.e., `x1`, `y1`, `psi1`) and controller outputs `TargetPoints` and `Deltas` from the MATLAB Workspace to a `.mat` file. Name the files so that you can easily identify them later.

Exercise 10.2: Straight-path experiments

`TruckLab_A2_Lab.m`

To evaluate the controller on a straight reference path, select one of the straight reference paths from the `path` folder in `TruckLab_A2_Lab.m` and carry out the experiment for the following initial conditions:

- place the tractor on the path with its heading aligned with the path

- place the tractor at an offset with its heading still perpendicular to the reference path
- place the tractor at an offset with its heading oriented towards the path
- place the tractor at an offset with its heading not facing the path

Throughout each experiment, observe the pop-up animation window and try to reason about the controller's behavior and relate it to the behavior of the physical, scaled RC tractor model.

Exercise 10.3: Sinusoidal-path experiments

To evaluate the controller on a sinusoidal reference path, select one of the sinusoidal reference paths in `TruckLab_A2_Lab.m` and carry out the experiment for the same initial conditions as described in the previous exercise.

Exercise 10.4: Plotting the results + Analysis

If you successfully completed all experiments in Exercises 10.2 and 10.3, you can now perform this exercise at home.

Your task is to generate the following figures:

- **Figure 1:** Plot the traversed route (OptiTrack data), the reference path, and the target points chosen by the algorithm in a single plot.
- **Figure 2:** Plot the steering angle computed by the controller and actuated on the tractor. Compare and reason about this signal with respect to the trajectory shown in **Figure 1**.
- **Figure 3:** Plot both the ATE and RMSE-ATE over time. For that purpose, reuse the MATLAB script `ex8_lookahead_dist_analysis.m`.

Analyze the behavior of the controller and explain your observations. Answer the following questions:

- Does the controller achieve similar tracking accuracy on both paths?
- Is the chosen lookahead distance equally suitable for both paths?
- What trade-off can be observed between smooth steering behavior and accurate path tracking?
- Can you observe the limitations of a Pure Pursuit controller?

Exercise 10.5: Robustness evaluation

Depending on how favorable the controller was towards your initial placement of the tractor, you may have observed poor path-following behavior. Using your understanding of the Pure Pursuit algorithm and the animation provided (which visualizes the controller's decisions), try to:

- Identify possible causes of such behavior.
- Propose improvements to the algorithm based on your observations.

References

Weisstein, E. W. (2025). *Circle–line intersection*. MathWorld–A Wolfram Web Resource.
<https://mathworld.wolfram.com/Circle-LineIntersection.html>